Business Analytics
Individual Assignment: Predicting Diabetes

NAMAN JAIN

ROLL NO - FT272022

SECTION 2

COURSE – Business Analytics

Overview

This report builds two classification models that predict whether a patient has diabetes, then compares them and recommends one for deployment. The dataset holds 768 patient records with 8 predictors and a binary target. 500 patients tested negative and 268 tested positive, so roughly 35% of the sample is diabetic.

Task A: data preparation

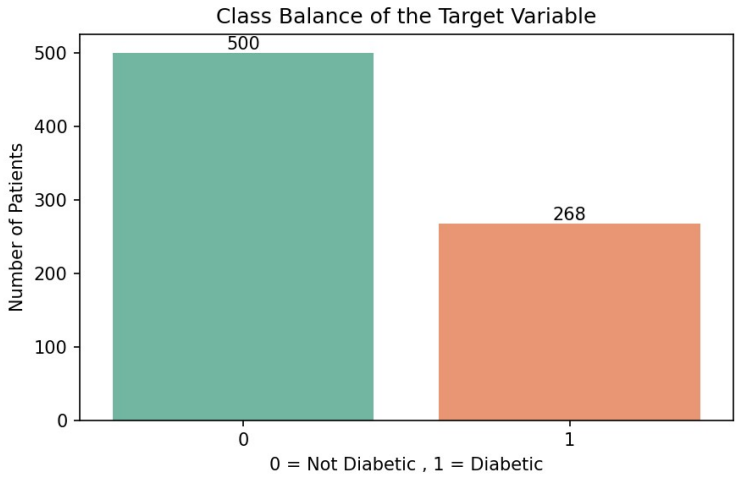
What the data looked like

All 9 columns are numeric and pandas reports no null values anywhere. There are no duplicate rows. On a first pass the file looks clean, which is exactly the trap the assignment is pointing at.

The summary statistics give it away. The minimum value of Glucose, Diastolic blood pressure, Triceps skinfold thickness, 2-Hour serum insulin and BMI is 0 in every one of those columns. A living patient cannot have a body mass index of 0 or a blood pressure of 0. These are missing measurements recorded as 0 rather than left blank, so pandas counts them as valid data.

Class balance of the target

500 patients tested negative and 268 positive, a 65:35 split. This shapes two later decisions. Accuracy on its own will mislead, because a model that labels everybody healthy already scores 64.9%. And both models will need class weighting so the smaller diabetic group is not drowned out.



Class balance of the target variable

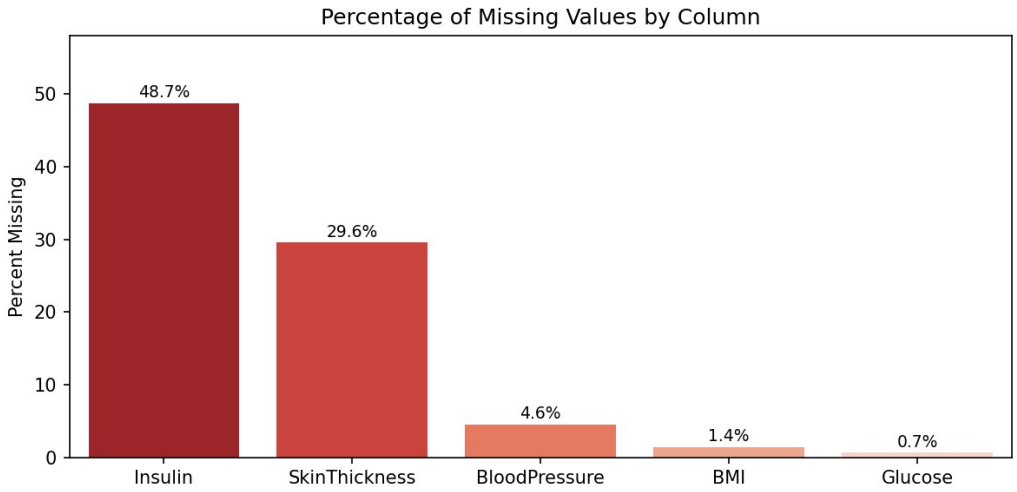
Which zeros are actually missing?

Counting the 0 column by column:

Column	Zeros	Share	Verdict
Insulin	374	48.7%	Missing
SkinThickness	227	29.6%	Missing
BloodPressure	35	4.6%	Missing
BMI	11	1.4%	Missing
Glucose	5	0.7%	Missing
Pregnancies	111	14.5%	Genuine value
Age	0	0.0%	No zeros

The Pregnancies column is the one that needs care. It has 111 zeros, but 0 pregnancies is a real and common value for a woman who has never been pregnant. Treating it as missing would have destroyed genuine information. To check this, we compared ages: the women recorded with 0 pregnancies average 27.6 years against 34.2 for the rest. Younger women, fewer pregnancies. The pattern is what you would expect, so those zeros stay.

That leaves the 5 columns the brief identifies. We replaced their 0 with NaN so that pandas would treat them as missing.



Share of values missing in each affected column, after the zeros are marked

Why we imputed instead of deleting

376 of the 768 rows carry at least 1 implausible 0. Dropping them would discard 49% of an already small dataset and leave 392 patients to train and test on. That is too expensive, so imputation was the only reasonable route.

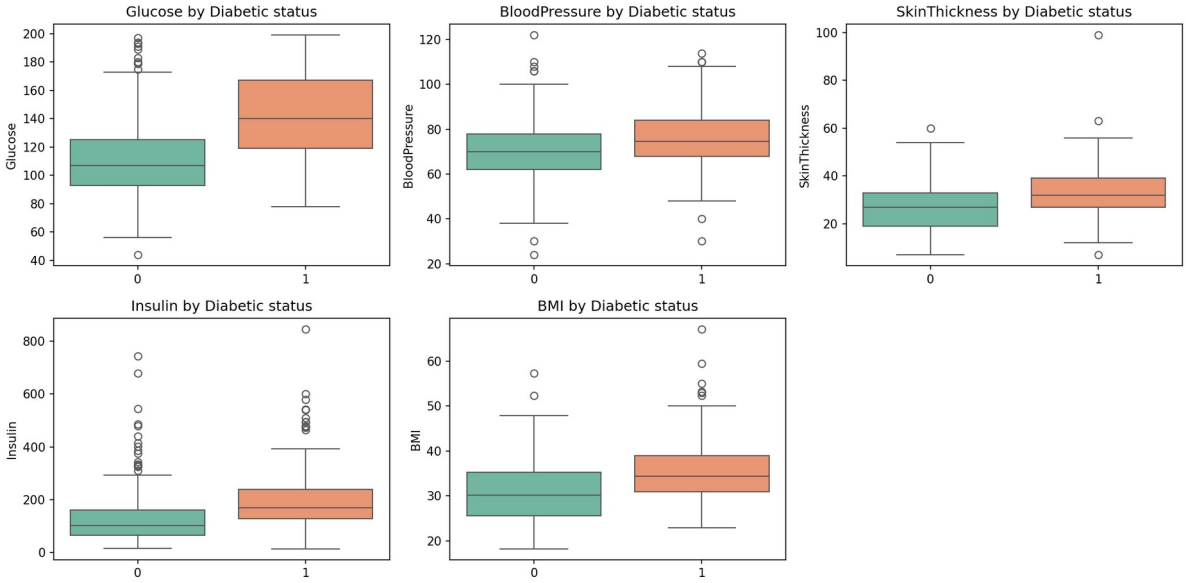
We filled the gaps with the median rather than the mean. Insulin and SkinThickness are both heavily right skewed, and the mean of a skewed variable gets dragged upward by the large values, which would have pushed the imputed patients toward the wrong end of the distribution.

A structural problem in the missingness

Cross tabulating which patients are missing what turned up something worth reporting. Every patient missing SkinThickness is also missing Insulin. There is not one case of the reverse: 227 patients are missing both, 147 are missing Insulin only, and 0 are missing SkinThickness only.

So, the gaps are not random. It looks like a group of patients never received the fuller workup that includes the skinfold and insulin measurements. Median imputation cannot repair that. It fills the hole with a plausible number but it cannot recover information that was never collected, and for nearly half the Insulin column we are effectively feeding the models a constant. This is carried forward as a limitation in the upcoming task.

How the variables separate the two classes



Distribution of each treated variable by diabetic status

Glucose separates the two groups far more cleanly than anything else, with diabetic patients sitting visibly higher. BMI and Age separate them too, though less sharply. Blood pressure barely separates them at all. This previews the feature importances in Section 7, and it is a useful sanity check: if the variable the models later lean on hardest showed no separation here, something would be wrong.

Train and test split

We split 70:30 with (random_state) set to 42 for reproducibility, giving 537 training and 231 test patients. The split is stratified on the target so both sets hold the same 65:35 class ratio. Without stratification an unlucky split can hand one set a noticeably different mix, and on a dataset this size that alone can move the metrics by a few points.

Imputation came after the split, not before

This ordering is deliberate and it is easy to get wrong. If the median is computed across the full dataset and then used to fill both sets, information from the test patients has leaked into the training data through that median. The test score is then no longer an honest estimate of performance on unseen patients, because those patients quietly helped shape the training inputs.

We therefore computed the medians on the 537 training rows only and applied those same numbers to the test set. The values used were 117 for Glucose, 72 for BloodPressure, 29 for SkinThickness, 126 for Insulin and 32.4 for BMI.

Task B: the Decision Tree

Establishing that overfitting is real

Before tuning anything we fitted a tree with no constraints, to see the problem rather than assume it. That tree grew to a depth of 13 with 98 leaves. It scored 100% accuracy on the training data and 69.3% on the test data.

A model that is perfect on data it has seen and worse than a coin flip weighted to the majority class on data it has not is memorising, not learning. With 537 training rows and 98 leaves, some leaves are describing a handful of individual patients. That gap is the whole reason the hyperparameters below exist.

Hyperparameters and the reasoning for each

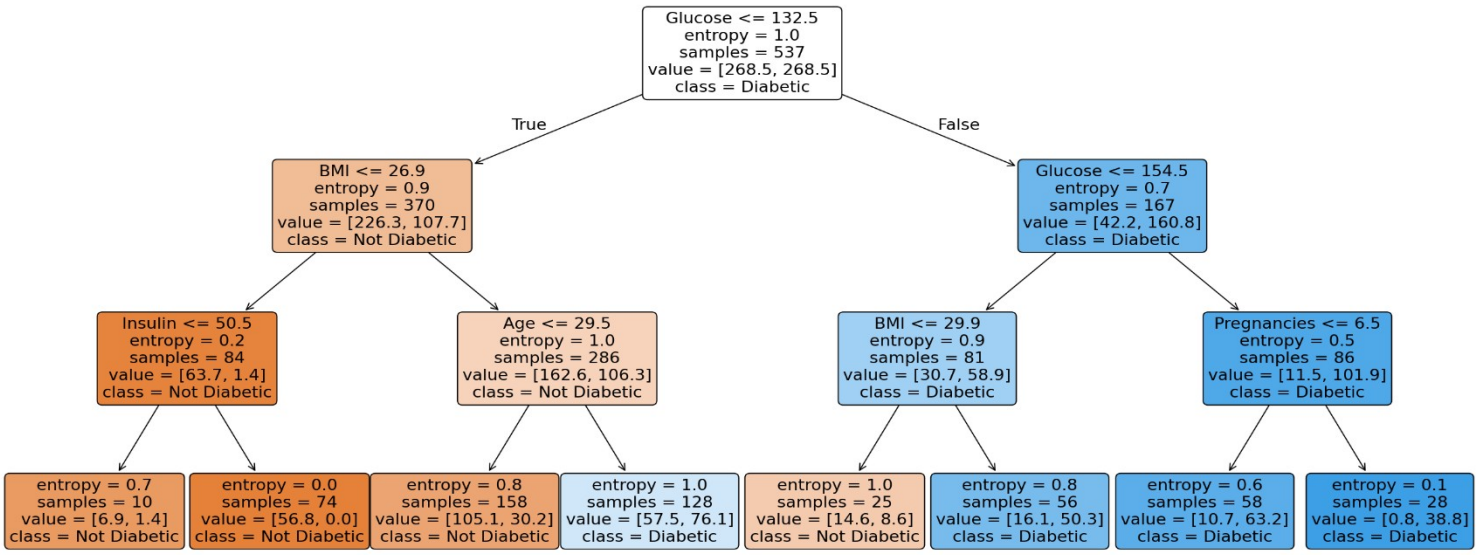
Settings were chosen by grid search with 5-fold stratified cross validation on the training set, scored on F1 for the diabetic class. The search selected the following.

Parameter	Value	Why
max_depth	3	The main brake on overfitting. Cuts depth from 13 to 3 and leaves from 98 to 8
min_samples_leaf	10	Every leaf must hold at least 10 patients, roughly 2% of the training set, so no leaf describes one unusual person
criterion	entropy	Selected by grid search over gini. In practice the two rarely differ much on this kind of data
class_weight	balanced	Weights the 268 diabetic cases up so the model does not drift toward always predicting the majority class

The class_weight setting is the one doing the most work here. Left unweighted, a model on a 65:35 split can score 65% accuracy by labelling everybody healthy, and gradient based search will happily walk toward that. Weighting the classes removes the incentive. Grid search picked it independently for both models, which suggests the imbalance genuinely needed handling rather than that we forced it.

After tuning, training accuracy is 74.1% and test accuracy is 74.5%. The two are within half a point of each other, so the memorisation is gone. Capping depth at 3 has the useful side effect that the tree fits on 1 page and a clinician could actually read the rules.

The fitted tree



The tuned Decision Tree, 3 levels deep with 8 leaves

The tree splits first on Glucose at 132.5, which sends 370 of the 537 training patients down the left branch and 167 down the right. On the left it then asks whether BMI is at or below 26.9, and on the right whether Glucose exceeds 154.5. The third level brings in Insulin, Age and Pregnancies.

Reading the leaves gives a rule a clinician could apply by hand. A patient with glucose above 154.5 and six or fewer pregnancies lands in a leaf of 28 training patients that is almost entirely diabetic. At the other end, glucose at or below 132.5 with a BMI at or below 26.9 and insulin above 50.5 produces a leaf of 74 patients with an entropy of zero, meaning every one of them tested negative. This readability is the single practical advantage the tree holds over the forest.

Test set performance

```
Accuracy  : 0.7446
Precision : 0.6038
Recall    : 0.7901
F1 score  : 0.6845
ROC AUC   : 0.8013

              precision    recall  f1-score   support

Not Diabetic       0.86      0.72      0.79       150
    Diabetic       0.60      0.79      0.68        81

    accuracy                           0.74       231
   macro avg       0.73      0.76      0.73       231
weighted avg       0.77      0.74      0.75       231

Confusion matrix :
[[108  42]
 [ 17  64]]
```

The tree catches 64 of the 81 diabetic patients in the test set and misses 17

Task C: the Random Forest

A random forest fits many trees on bootstrapped samples of the data and lets them vote. Each split considers only a random subset of the features, which stops all the trees from becoming copies of one another. Averaging across many noisy trees cancels out much of the variance that made the single unrestricted tree fail.

Parameter	Value	Why
n_estimators	300	More trees give a steadier answer. A forest does not overfit by adding trees, it just stops improving, and scores had already flattened by 300
max_depth	6	Deeper than the single tree is safe here, because averaging across 300 trees absorbs variance one deep tree could not
min_samples_leaf	5	Stops individual trees splitting down to near single patients.
max_features	sqrt	Each split sees about 3 of the 8 features. This is what decorrelates the trees. If every tree saw every feature they would all split on Glucose first and the forest would be one tree repeated 300 times
class_weight	balanced	Same reasoning as the Decision Tree

Training accuracy is 84.9% against 75.3% on test. That gap is wider than the tree's, so the forest is fitting the training data harder. For a bagged ensemble this is normal rather than alarming, and the cross validation in Section 6 confirms the test score is stable.

Test set performance

```
Accuracy  : 0.7532
Precision : 0.625
Recall    : 0.7407
F1 score  : 0.678
ROC AUC   : 0.8441

              precision    recall  f1-score   support

Not Diabetic       0.84      0.76      0.80       150
    Diabetic       0.62      0.74      0.68        81

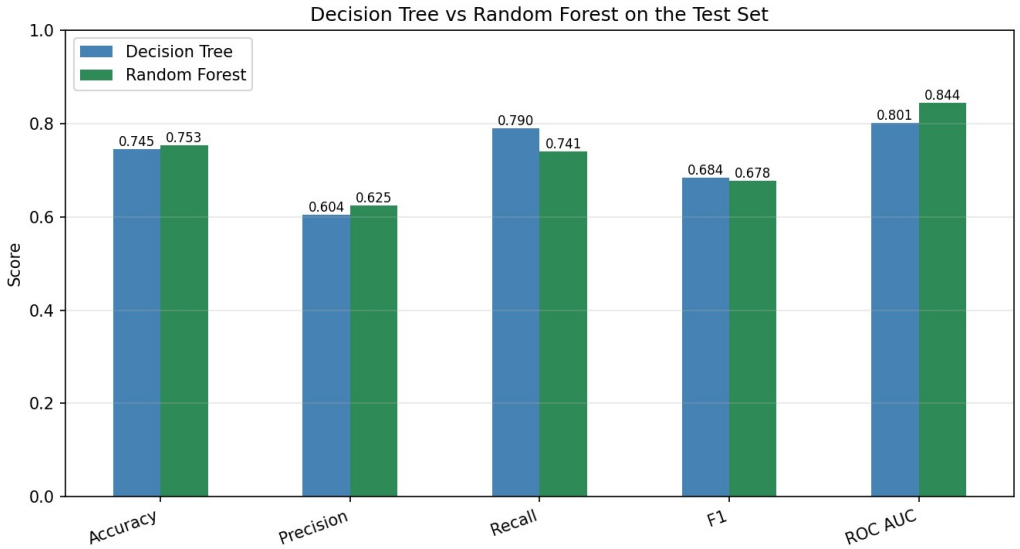
    accuracy                           0.75       231
   macro avg       0.73      0.75      0.74       231
weighted avg       0.77      0.75      0.76       231

Confusion matrix :
[[114  36]
 [ 21  60]]
```

Task D: comparing the two models

Both models were evaluated on the same 231 held out patients using the same procedure. All precision, recall and F1 figures are for the positive (diabetic) class.

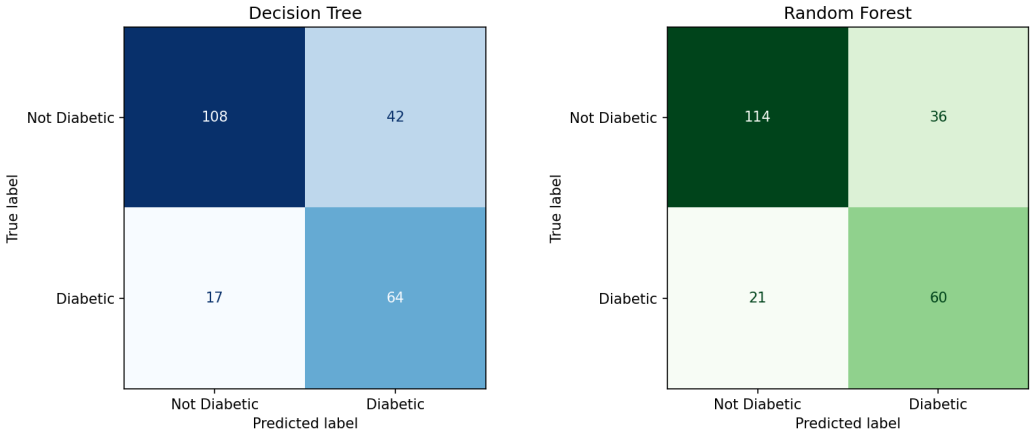
Metric	Decision Tree	Random Forest	Winner
Accuracy	0.7446	0.7532	Forest
Precision (diabetic)	0.6038	0.6250	Forest
Recall (diabetic)	0.7901	0.7407	Tree
F1 (diabetic)	0.6845	0.6780	Tree
ROC AUC	0.8013	0.8441	Forest



Test set metrics side by side. The tree leads on recall, the forest on the rest

And the confusion matrices, which matter more than the headline scores in a screening context:

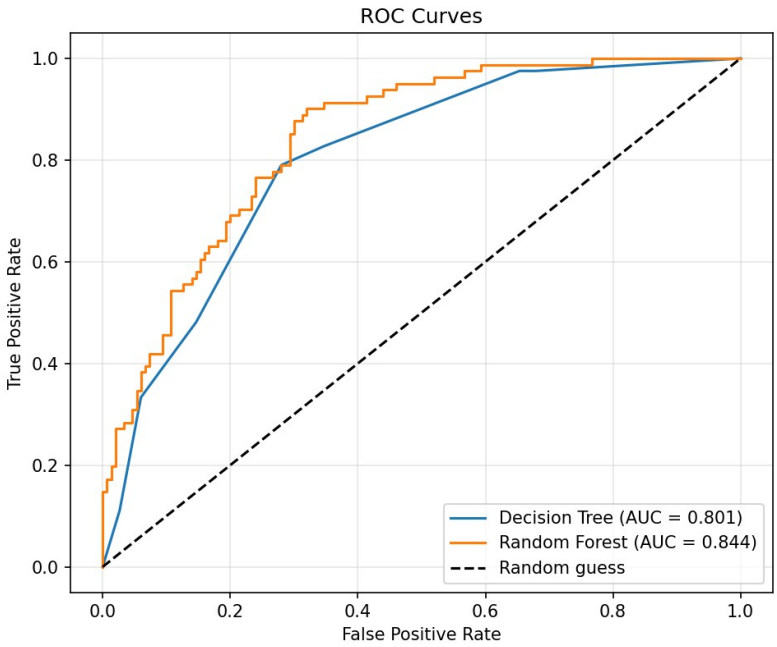
Outcome	Decision Tree	Random Forest
True positives (diabetic, caught)	64	60
True negatives (healthy, cleared)	108	114
False positives (false alarm)	42	36
False negatives (patient missed)	17	21



Confusion matrices. The bottom left cell of each is the missed patients

For context, a model that simply labelled every patient healthy would score 64.9% accuracy on this test set. Both models beat that, but not by as much as the raw accuracy figures might suggest, which is the usual reason accuracy is a poor headline metric on imbalanced data.

Performance across every threshold



ROC curves. The forest covers more area, so it ranks patients by risk more accurately

The table above compares the models at one fixed cut off. The ROC curve compares them at every possible cut off, and here the forest is clearly ahead with an area of 0.844 against 0.801. The 2 curves cross at the left, which is why the models look so similar on the fixed-threshold metrics, but the forest holds a better true positive rate across most of the range. That gap is the opening which next part exploits.

Is the difference real, or an artefact of one split?

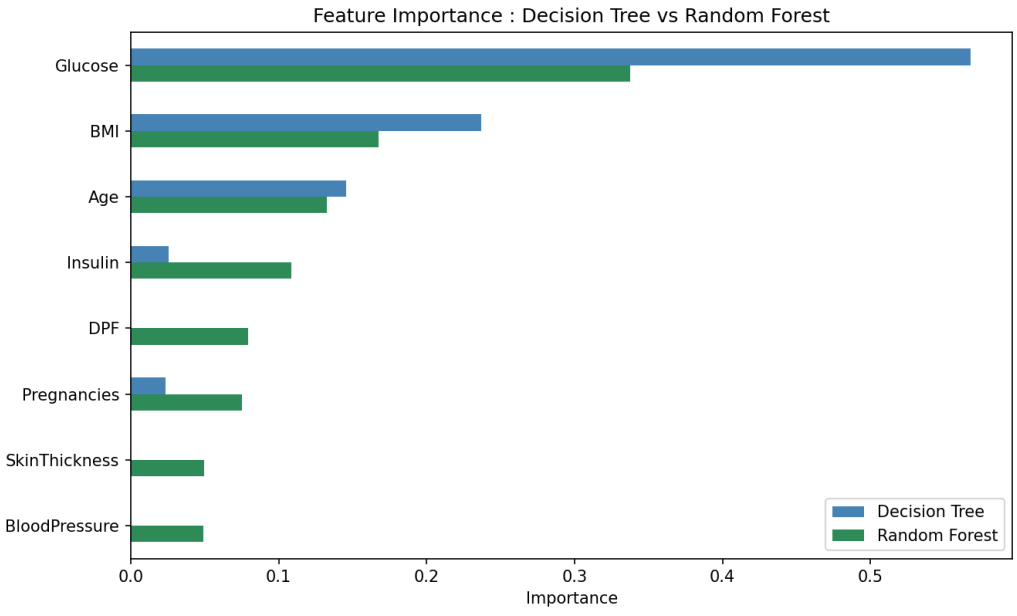
231 test patients is a small sample and a single split can flatter either model by chance. We therefore ran 5 fold cross validation on the training set as a check.

Metric	Decision Tree	Random Forest
Accuracy	0.7170 (sd 0.034)	0.7618 (sd 0.039)
Recall	0.8233 (sd 0.037)	0.7172 (sd 0.085)
F1	0.6704 (sd 0.025)	0.6766 (sd 0.048)
ROC AUC	0.8151 (sd 0.018)	0.8379 (sd 0.040)

The pattern from the test set holds up. The tree's recall advantage is real and it is also steadier, with a standard deviation of 0.037 against the forest's 0.085. The forest's advantage on accuracy and AUC is likewise consistent. So, the trade off is a genuine property of the two models on this data, not a quirk of the split we happened to draw.

Task E: feature importances

Feature	Decision Tree	Random Forest
Glucose	0.5678	0.3378
BMI	0.2369	0.1674
Age	0.1458	0.1327
Insulin	0.0258	0.1088
Diabetes pedigree function	0.0000	0.0793
Pregnancies	0.0237	0.0752
SkinThickness	0.0000	0.0498
BloodPressure	0.0000	0.0491



Feature importance from both models. The forest spreads weight across all eight variables

Glucose dominates both models, which is reassuring rather than surprising. The target was defined by an oral glucose tolerance test, and plasma glucose is measured during that same test, so the strongest predictor is closely related to the diagnostic procedure itself. BMI and Age follow, and both are established risk factors.

The more interesting detail is the disagreement. The Decision Tree assigns exactly 0 importance to the diabetes pedigree function, skinfold thickness and blood pressure. It would be a mistake to read that as those variables carrying no information. With max_depth capped at 3 the tree has room for only 7 splits in total, so once it has used Glucose, BMI and Age there is nothing left to spend. Anything not selected scores 0 by construction.

The forest tells a fairer story precisely because max_features forces most trees to split without Glucose available. Every variable gets a non-zero share, and the diabetes pedigree function in particular jumps from nothing to 0.079. The practical lesson is that a single shallow tree's importance ranking says as much about the depth limit as it does about the data.

Which model to deploy??

The costs are not symmetric

Both models label a patient diabetic when the predicted probability passes 0.5. That default carries a hidden assumption: that a false positive and a false negative cost the same. In a screening programme they clearly do not.

A false positive sends a healthy patient for a confirmatory blood test. That costs a little money and causes some worry, and the follow up test corrects the error. A false negative tells a diabetic patient they are fine. They leave the screening believing they are healthy, the condition goes untreated, and there is no second check waiting to catch the mistake. In a condition where early management materially changes outcomes, those two errors are not close to equivalent.

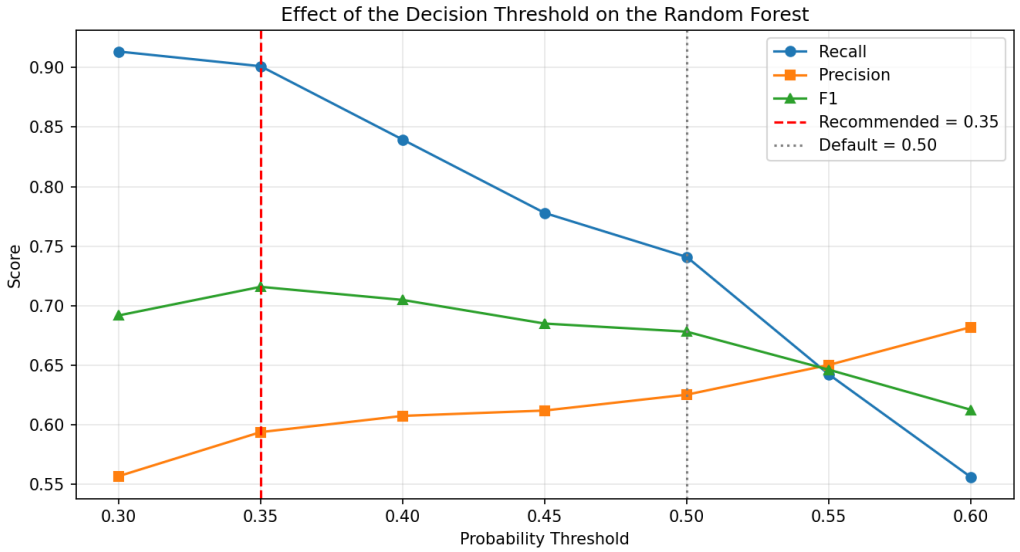
Since screening is a first filter rather than a diagnosis, the model should be tuned to miss as few patients as possible and let the confirmatory test remove the false alarms.

Why the threshold matters more than the algorithm

The forest's higher AUC (0.844 against 0.801) says it ranks patients by risk better than the tree does, even though the 2 score similarly at the 0.5 cut off. AUC measures ranking quality across every possible threshold, so a better AUC means there is more to gain by moving the threshold. That is the opening worth taking.

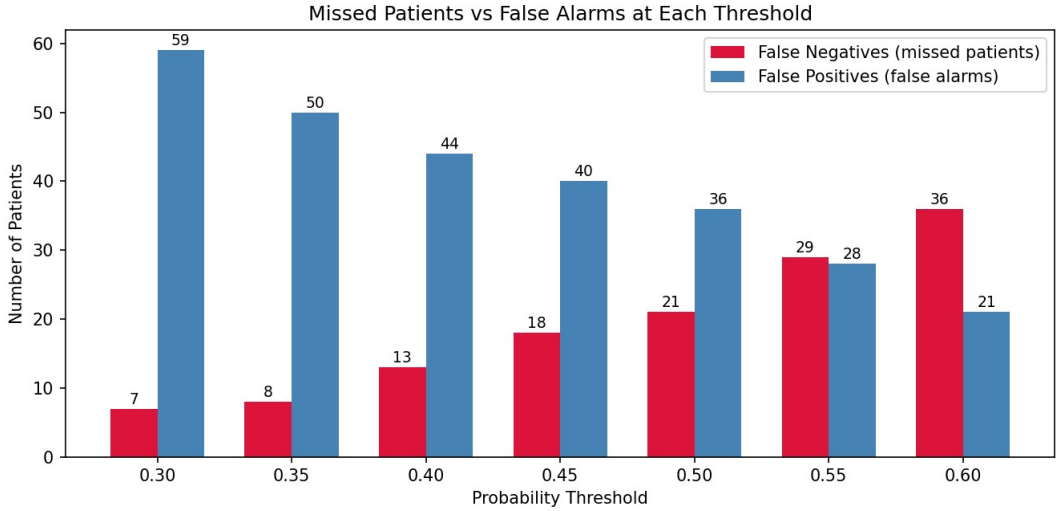
Lowering the forest's cut off produces this:

Threshold	Accuracy	Precision	Recall	F1	Missed	False alarms
0.30	0.7143	0.5564	0.9136	0.6916	7	59
0.35	**0.7489**	**0.5935**	**0.9012**	**0.7157**	**8**	**50**
0.40	0.7532	0.6071	0.8395	0.7047	13	44
0.45	0.7489	0.6117	0.7778	0.6848	18	40
0.50 (default)	0.7532	0.6250	0.7407	0.6780	21	36
0.55	0.7532	0.6500	0.6420	0.6460	29	28
0.60	0.7532	0.6818	0.5556	0.6122	36	21



How precision, recall and F1 move as the cut off shifts. F1 peaks at 0.35

At a threshold of 0.35 the forest catches 73 of the 81 diabetic patients instead of 60. Missed patients fall from 21 to 8. The cost is 14 additional false alarms, 50 instead of 36, which in practice means 14 more confirmatory blood tests across 231 patients.



The trade being made. Missed patients fall steeply as the threshold drops, false alarms rise gently

This chart is the clearest statement of the decision. Reading right to left, every step down in threshold buys back missed patients at a modest cost in false alarms. Below 0.35 the return flattens: going to 0.30 saves one further patient but adds nine more false alarms, which is where we stop.

F1 improves at the same time, from 0.678 to 0.716, which is the highest F1 of any configuration tested including both models at their defaults. Accuracy is essentially unchanged at 74.9%. So this is not a case of trading away overall quality to chase recall. The default threshold was simply not the right operating point for this problem.

RANDOM FOREST WITH THRESHOLD 0.35 - FINAL RECOMMENDED MODEL

```
Accuracy  : 0.7489
Precision : 0.5935
Recall    : 0.9012
F1 score  : 0.7157

              precision    recall  f1-score   support

Not Diabetic       0.93      0.67      0.78       150
    Diabetic       0.59      0.90      0.72        81

    accuracy                           0.75       231
   macro avg       0.76      0.78      0.75       231
weighted avg       0.81      0.75      0.75       231

Confusion matrix :
[[100  50]
 [  8  73]]
```

Recommendations

Deploy the Random Forest at a decision threshold of 0.35

	Decision Tree	Random Forest @ 0.50	Random Forest @ 0.35
Accuracy	0.7446	0.7532	0.7489
Precision	0.6038	0.6250	0.5935
Recall	0.7901	0.7407	0.9012
F1	0.6845	0.6780	0.7157
Patients missed	**17**	**21**	**8**

The reasoning runs in 3 steps. The forest ranks risk better than the tree, which the AUC gap establishes. Better ranking is what makes threshold tuning pay off. And at 0.35 the tuned forest beats both defaults on recall and on F1 simultaneously, while giving up almost nothing on accuracy.

The Decision Tree is a single readable diagram, and a clinician can follow its logic without trusting a black box, which matters for adoption in a medical setting. If interpretability were a hard requirement, the tree at depth 3 would be a defensible choice and it still reaches 0.79 recall. Our judgement is that missing 17 patients instead of 8 is too high a price for that transparency, particularly since the forest's individual predictions can be explained after the fact if needed.

Limitations

- Nearly half the Insulin column and 30% of SkinThickness were imputed with a single constant. For those patients the model is not using a real measurement, and the 2 columns are missing together in a structured way rather than at random, so imputation cannot recover what was never recorded.
- 768 records is small for this kind of work, and the test set is 231 patients. A single false negative moves recall by more than a percentage point, so differences of one or two points between models should not be read as meaningful.
- Glucose is measured during the same oral glucose tolerance test that defines the target. Its dominance partly reflects that closeness rather than pure predictive insight, and the model would be less impressive if asked to predict diabetes without any glucose measurement.
- The data comes from a specific population, women of Pima Indian heritage aged 21 and over. Diabetes prevalence in that group is unusually high, so both the base rate and the learned relationships may not carry over to a general screening population.

- The 0.35 threshold was chosen by looking at the test set. Strictly it should be selected on a separate validation split, otherwise the reported test performance at that threshold is mildly optimistic.

What would improve this most

The single highest value fix is to collect the missing measurements properly rather than impute them. Nearly half the Insulin values are absent, and the fact that they go missing together with SkinThickness points at a specific gap in the data collection protocol rather than scattered human error. Finding and fixing that gap would give the models real information for those patients instead of a repeated median.

Beyond that, two things would help. Adding features the dataset does not carry, such as family history detail, diet, physical activity or HbA1c, would give the model something to work with when glucose is borderline. And holding out a proper validation set for threshold selection would let us quote an unbiased figure for the deployed configuration rather than the mildly optimistic one above.

Conclusion

Neither model is clearly better at the default cut off. The decision tree catches more diabetic patients and edges the F1 score. The Random Forest is more accurate, more precise, and ranks patients by risk considerably better - AUC 0.844 against 0.801. Cross validation confirms both halves of that trade off are stable rather than accidental.

Glucose, BMI and Age are the variables that carry the prediction, and the two models agree on that ordering even though they disagree on how much weight the remaining five deserve.

The finding that changed the recommendation was not about the algorithms at all. Once the asymmetry between a missed diagnosis and a false alarm is taken seriously, the default 0.5 threshold is the wrong operating point, and moving the forest to 0.35 cuts missed patients from 21 to 8 while improving F1 and leaving accuracy essentially untouched. Choosing the right threshold turned out to matter more than choosing between a tree and a forest.

References

Sources consulted while preparing this analysis. Course materials are listed first, followed by library documentation and the published sources for the dataset and the two algorithms.

Course materials -

Business Analytics course notes on clustering, decision trees and CART, covering Gini and entropy splitting criteria and tree pruning. Lecture notes, taken in class.

Data Mining, Notebook 1: Hierarchical Clustering. Course notebook supplied by the instructor. Referenced for coding conventions and workflow structure.

Data Mining, Notebook 1: K-Means Clustering. Course notebook supplied by the instructor. Referenced for the cross validation loop pattern and the profiling approach used in Step 11.

Software documentation -

scikit-learn developers. sklearn.tree.DecisionTreeClassifier, sklearn.ensemble.RandomForestClassifier, sklearn.model_selection.train_test_split and sklearn.metrics

Official API documentation, https://scikit-learn.org/stable/ - Consulted for parameter definitions and default behaviour

https://pandas.pydata.org/docs/ - Consulted for DataFrame.apply, DataFrame.replace and groupby behaviour

https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained